AGENTIC AI WORKSHOP · DAY 4 OF 10

LangGraph: state, memory & human approval

Meridian becomes a state machine: branching, persistence, long-term memory, and a human approval gate before it raises a limit.

You build: **Approval gate** · about 4 h of material

Contents

1. Payment firms adopt agents that stop and ask — Why this matters · 5 min
2. Agents as graphs — Theory · 10 min
3. State, reducers and memory — Theory · 10 min
4. Lab 1: Meridian as a graph — Lab · beginner · 30 min
5. Lab 2: Conversations that survive a restart — Lab · intermediate · 25 min
6. Human-in-the-loop: interrupt, approve, resume — Theory · 10 min
7. Lab 3: The approval gate — Lab · intermediate · 35 min
8. Lab 4: Memory across sessions — Lab · intermediate · 20 min
9. Parallel work: Send, reducers and map-reduce — Theory · advanced · 10 min
10. Lab 5: The month-end review, in parallel — Lab · advanced · 20 min
11. Durable execution: retries, timeouts, caching, resuming — Theory · advanced · 10 min
12. Lab 6: A settlement run that survives a crash — Lab · advanced · 20 min
13. Time travel: replay and fork — Theory · advanced · 5 min
14. Lab 7: What if the limit were higher? — Lab · advanced · 15 min
15. Taking a graph to production — Theory · advanced · 5 min
16. Check yourself — Quiz · 5 min
17. Day 4 recap — Recap · 3 min
18. Quiz answer key

WHY THIS MATTERS 5 MIN

Payment firms adopt agents that stop and ask

Agents don't get adopted in payments because they're smart. They get adopted because they pause before moving money.

ACME is over its limit (9.92M vs 9.5M USD) and Black Friday starts tomorrow. The account manager asks Meridian to **"raise ACME's limit to 10.5M for 5 days"**. Meridian may **propose** this, but a merchant risk officer must **approve** it.

That's the **four-eyes principle**: the same control payment firms already apply to people. Today you build it into the agent, in code, so it can't be skipped.

By the end of today you can

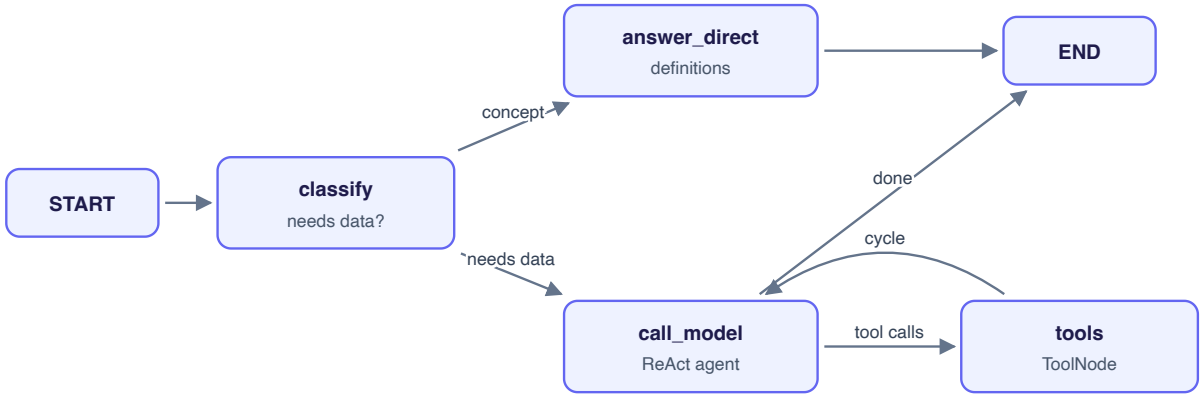
- Build an agent as an explicit graph with branches and a loop
- Explain state, reducers and the three kinds of memory
- Persist conversations so they survive a restart
- Pause for human approval, then resume with approve / edit / reject
- Give the agent memory that lasts across sessions

THEORY 10 MIN

Agents as graphs

Day 3's create_agent hid the loop. A graph makes it explicit, so you can add a branch, a check or a pause exactly where you want.

Meridian as a graph



1. Every question starts at classify: does it need Northstar data, or is it a definition?
2. 'What is a rolling reserve?' needs no data, so answer it directly. Cheap and fast.
3. 'Is ACME over its limit?' goes to the ReAct agent.
4. The Day 2 loop is now two nodes and a cycle: call_model ⇄ tools.
5. No more tool calls → END. recursion_limit plays the role of MAX_TURNS.

Concept	Meaning
State	a typed dict passed between nodes (`messages`, `route`, ...)
Node	a function: state → partial update
Edge / conditional edge	a fixed transition / a function that picks the next node
Cycle	an edge back to an earlier node: the agent loop

Key points

- Put business rules in edges (e.g. 'every limit increase goes through approval'): deterministic and testable.
- Keep nodes small so traces read like a flowchart.

THEORY 10 MIN

State, reducers and memory

A reducer decides how a node's update merges into the state. Memory comes in three lifetimes.

```
class MeridianState(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]   # reducer: APPEND
    route: Literal["direct", "agent"] | None              # no reducer: last write wins
```

Memory	Lasts	LangGraph tool	Meridian example
Working	one run	graph **state**	the FX rate fetched this run
Short-term	one conversation	**checkpointer** + `thread_id`	"and what about BOLT?" follow-up
Long-term	across conversations	**store** + namespace	"this analyst wants GBP figures"

Trim what the model sees, keep the full record. Long sessions blow the context window and the budget, so trim the messages you send each call, but don't rewrite stored history.

LAB BEGINNER 30 MIN

Lab 1: Meridian as a graph

Rebuild Day 3's agent as an explicit graph, so you can see (and later change) every step it takes.

Goal. An explicit graph with a classify branch and the ReAct cycle, that you can draw, stream and debug node by node.

1. Install LangGraph

LangGraph 1.x is the stable line. `langgraph-checkpoint-sqlite` gives you persistence (Lab 2) and a store (Lab 4). Check with `pip show langgraph`: you want 1.2 or newer.

RUN
```
pip install "LANGGRAPH>=1.2,<2" "LANGGRAPH-CHECKPOINT-SQLITE>=3.1,<4"
```

2. Build the graph

Read it top to bottom, in this order:

1. **State** (`MeridianState`): `messages` has the `add_messages` reducer, so every node *appends*; `route` has none, so the last write wins.
2. **Nodes** are plain functions `state → partial update`. `classify` uses **structured output** (`Route`), so routing is a typed value, not a string you have to parse.
3. **Edges**: `add_conditional_edges("classify", lambda s: s["route"], {...})` maps the value to the next node. `tools_condition` sends the model to `tools` if it asked for any, otherwise to `END`.
4. **The cycle**: `tools → call_model`. That single edge *is* the agent loop from Day 2.

`graph.py`

```python
# graph.py
from dotenv import load_dotenv
load_dotenv()  # reads OPENROUTER_API_KEY from .env

from typing import Annotated, Literal
from typing_extensions import TypedDict
from pydantic import BaseModel
from langchain_openrouter import ChatOpenRouter      # Claude via OpenRouter
from langchain_core.messages import AnyMessage, SystemMessage
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode, tools_condition
from tools_lc import TOOLS                    # Day 3 @tool functions

class MeridianState(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]
    route: Literal["direct", "agent"] | None

llm = ChatOpenRouter(model="anthropic/claude-opus-5.5", max_tokens=16000)
agent_llm = llm.bind_tools(TOOLS)

SYSTEM = SystemMessage("You are Meridian, a merchant-risk & payments ops analyst at Northstar Pay. "
                       "Convert all amounts into the limit currency. Use the calculator for
arithmetic.")

# --- node 1: classify (structured output → deterministic routing) ---
class Route(BaseModel):
    route: Literal["direct", "agent"]
```

```
    reason: str

classifier = llm.with_structured_output(Route)

def classify(state: MeridianState):
    r = classifier.invoke([SystemMessage(
        "Route to 'direct' if the question is a definition/concept answerable without Northstar data.
"
        "Route to 'agent' if it needs merchant data, FX rates, or calculations."), state["messages"]
[-1]])
    return {"route": r.route}

# --- node 2a: direct answer ---
def answer_direct(state: MeridianState):
    return {"messages": [llm.invoke([SYSTEM, *state["messages"]])]}

# --- node 2b: agent (ReAct) ---
def call_model(state: MeridianState):
    return {"messages": [agent_llm.invoke([SYSTEM, *state["messages"]])]}

builder = StateGraph(MeridianState)
builder.add_node("classify", classify)
builder.add_node("answer_direct", answer_direct)
builder.add_node("call_model", call_model)
builder.add_node("tools", ToolNode(TOOLS))     # executes tool calls, returns ToolMessages, handles
errors

builder.add_edge(START, "classify")
builder.add_conditional_edges("classify", lambda s: s["route"],
                              {"direct": "answer_direct", "agent": "call_model"})
builder.add_conditional_edges("call_model", tools_condition)   # → "tools" or END
builder.add_edge("tools", "call_model")                        # the cycle
builder.add_edge("answer_direct", END)

graph = builder.compile()
print(graph.get_graph().draw_mermaid())
```

3. Run both routes

The rolling-reserve question needs no data, so it should never touch a tool. The ACME question must. If both go to `agent`, read the classifier prompt: is the definition rule clear?

run

```
for q in ["What is a rolling reserve?",                            # → direct
          "Is ACME over its processing limit?"]:                   # → agent + tools
    out = graph.invoke({"messages": [("user", q)]}, {"recursion_limit": 20})
    print(out["route"], "→", out["messages"][-1].text[:300])
```

4. Watch the nodes fire

`stream_mode="updates"` yields one dict per node, as it finishes. You'll see something like `['classify']` `['call_model']` `['tools']` `['call_model']` `['tools']` `['call_model']`: each `call_model ⇄ tools` pair is one turn of the loop. Other modes: `"values"` (whole state after each step), `"messages"` (LLM tokens as they're generated, for chat UIs), `"custom"` (your own events, Lab 5).

stream

```
for step in graph.stream({"messages": [("user", "Is CRUX over its limit?")]}, stream_mode="updates"):
    print(list(step.keys()))     # ['classify'] ['call_model'] ['tools'] ['call_model'] ...
```

5. **Draw it**

   `print(graph.get_graph().draw_mermaid())` and paste the output into **mermaid.live**. Check that the picture matches what you think the code does: dashed lines are conditional edges.

6. **Break it on purpose**
   - Set `{"recursion_limit": 3}` in the config and ask about CRUX. You get **GraphRecursionError**: LangGraph's version of Day 2's `MAX_TURNS`. In production, catch it and return "I couldn't finish this; a person will follow up" rather than a stack trace.
   - Remove the `tools → call_model` edge and run again. The agent calls a tool and then… stops. That edge is the loop.

Expected result. The rolling-reserve question goes classify → answer_direct; ACME goes classify → call_model ⇄ tools → END, and the answer says OVER_LIMIT at about 104.4%.

LAB INTERMEDIATE 25 MIN

Lab 2: Conversations that survive a restart

A checkpointer saves the state after every step. That one line gives you memory, resumability and an audit trail.

Goal. Add a SQLite checkpointer so a follow-up question works in a brand-new process, then inspect and replay the saved history.

1. Compile with a checkpointer

- `thread_id` is the conversation's ID. Same thread = same conversation; a new ID starts fresh.
- The checkpointer writes after **every super-step** (each round of nodes), not just at the end. That's what makes Lab 6 (crash recovery) and Lab 7 (time travel) possible.
- `SqliteSaver` is for development and single machines. In production use `PostgresSaver` (`langgraph-checkpoint-postgres`): same API.

persist.py

```
# persist.py
from langgraph.checkpoint.sqlite import SqliteSaver
from graph import builder                    # Lab 1 builder (uncompiled)

with SqliteSaver.from_conn_string("meridian.db") as checkpointer:
    graph = builder.compile(checkpointer=checkpointer)
    cfg = {"configurable": {"thread_id": "analyst-42-session-1"}, "recursion_limit": 20}

    graph.invoke({"messages": [("user", "What is ACME's MTD volume vs limit?")]}, cfg)
    out = graph.invoke({"messages": [("user", "And how much headroom would a 1M USD transaction
leave?")]}, cfg)
    print(out["messages"][-1].text)
```

2. Prove persistence

Run the first question, **exit Python completely**, then run only the follow-up with the same `thread_id`. Meridian still knows you meant ACME. Change `thread_id` and it doesn't: **thread = session**.

3. Inspect and time-travel

Every super-step is a checkpoint: a gift for auditors ("what did the agent know when it said that?").

history

```
state = graph.get_state(cfg)
print(state.next, len(state.values["messages"]))

for snap in graph.get_state_history(cfg):          # newest first, one per super-step
    print(snap.config["configurable"]["checkpoint_id"], snap.next, len(snap.values["messages"]))

# Fork from an earlier checkpoint (e.g. replay with a corrected input)
old = list(graph.get_state_history(cfg))[3]
graph.invoke(None, old.config)                      # resumes from that point
```

4. Look inside the database

Checkpoints are just rows. That's why you must treat this file like any other customer data: it contains the full conversation, including tool results.

RUN

```
sqlite3 meridian.db ".TABLES"
sqlite3 meridian.db "SELECT THREAD_ID, COUNT(*) FROM CHECKPOINTS GROUP BY THREAD_ID;"
```

5. Trim what the model sees

Long sessions grow the message list without limit. Keep the full history in the checkpoint, but send only recent turns to the model:

```
from langchain_core.messages.utils import trim_messages, count_tokens_approximately

def call_model(state):
    recent = trim_messages(state["messages"], strategy="last", max_tokens=6000,
                           token_counter=count_tokens_approximately, start_on="human",
include_system=True)
    return {"messages": [agent_llm.invoke([SYSTEM, *recent])]}
```

`start_on="human"` matters: cutting between a tool call and its result makes the API reject the request.

Expected result. The follow-up answer refers to ACME without you naming it again, and the history shows one checkpoint per step with `next` pointing at the node that would run from there.

THEORY 10 MIN

Human-in-the-loop: interrupt, approve, resume

interrupt() saves the state and ends the run. Resuming is a new call, possibly hours later, from another process.

An approval gate on increase_processing_limit



1. The model asks for increase_processing_limit. A conditional edge routes sensitive tools to the approval node.
2. interrupt() pauses the graph and saves the state. The payload (merchant, new limit, days) goes to a human.
3. Approve, or EDIT the arguments (10.5M → 10M, 3 days), and the tool runs.
4. Reject: the model gets 'REJECTED: <reason>' and must answer without making the change.

Key points

- interrupt() needs a checkpointer and a stable thread_id.
- On resume the node re-runs from the top: put side effects AFTER interrupt(), or make them safe to repeat.
- Start by approving everything, measure, then auto-approve the low-risk band (≤5%) in code.

LAB INTERMEDIATE 35 MIN

Lab 3: The approval gate

The model may propose a limit increase. Only a person can approve it, and the rule lives in code where the model can't talk its way around it.

Goal. Meridian proposes a limit increase; the run pauses; a merchant risk officer approves, edits or rejects it, possibly hours later, from another process.

1. The sensitive (mock) tool

It changes a limit (here, in memory). In real life this would call the risk system's API. `SENSITIVE` is the list of tools that always need a person: a set in code, not a sentence in a prompt.

tools_sensitive.py

```
# tools_sensitive.py
from langchain_core.tools import tool
from data import MERCHANTS

@tool
def increase_processing_limit(merchant_id: str, new_limit: float, currency: str, days: int,
justification: str) -> str:
    """Request a TEMPORARY processing limit increase. Requires merchant-risk-officer approval. Max 30
days."""
    cp = MERCHANTS[merchant_id.upper()]
    old = cp["processing_limit"]["amount"]
    cp["processing_limit"]["amount"] = new_limit          # mock side effect
    return f"Limit for {merchant_id} changed {old:,.0f} → {new_limit:,.0f} {currency} for {days}
days."

SENSITIVE = {"increase_processing_limit"}
```

2. The graph with an approval node

The three parts to understand:

1. `route_after_model` looks at the tool calls. Any sensitive one → `approval`; otherwise → `tools`. This is deterministic code.
2. `interrupt(payload)` saves the state and stops the run. The payload is what the approver sees: action, arguments and the policy.
3. **On resume**, `interrupt()` returns the approver's decision and the node runs again **from the top**. So never put side effects (sending an email, writing a log) *before* `interrupt()`: they'd happen twice.

`Command(goto=..., update=...)` lets a node choose the next node *and* change the state in one step. That's how "edit" rewrites the tool call's arguments.

hitl_graph.py

```
# hitl_graph.py — Meridian with an approval gate. The model can PROPOSE a limit increase; a person
decides.
from typing import Annotated
from typing_extensions import TypedDict
from dotenv import load_dotenv
from langchain_core.messages import AnyMessage, SystemMessage, ToolMessage
from langchain_openrouter import ChatOpenRouter
from langgraph.graph import END, START, StateGraph
```

```python
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode
from langgraph.types import Command, interrupt
from tools_lc import TOOLS                          # Day 3: calculator, lookup_merchant, get_fx_rate
from tools_sensitive import SENSITIVE, increase_processing_limit

load_dotenv()

class State(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]

ALL_TOOLS = TOOLS + [increase_processing_limit]
llm = ChatOpenRouter(model="anthropic/claude-opus-5.5", max_tokens=8000).bind_tools(ALL_TOOLS)
SYSTEM = SystemMessage(
    "You are Meridian, a merchant-risk analyst at Northstar Pay. Use tools for every fact and number.
"
    "You may PROPOSE a temporary limit increase with increase_processing_limit; a merchant risk
officer decides."
)

def call_model(state: State):
    return {"messages": [llm.invoke([SYSTEM, *state["messages"]])]}

def route_after_model(state: State):
    """Business rule in CODE, not in the prompt: sensitive tools always go to a person first."""
    last = state["messages"][-1]
    if not last.tool_calls:
        return END
    return "approval" if any(c["name"] in SENSITIVE for c in last.tool_calls) else "tools"

def approval(state: State) -> Command:
    last = state["messages"][-1]
    call = next(c for c in last.tool_calls if c["name"] in SENSITIVE)
    # ⏸ interrupt() saves the state and stops the run. The value we pass is what the approver sees.
    # When the run is resumed with Command(resume=...), interrupt() RETURNS that value and the node
re-runs from the top.
    decision = interrupt({
        "action": call["name"],
        "args": call["args"],
        "policy": "Increases need a merchant risk officer. Max 30 days.",
    })
    if decision["type"] == "approve":
        return Command(goto="tools")
    if decision["type"] == "edit":                  # the approver changes the arguments, e.g. a
smaller increase
        edited = {**call, "args": {**call["args"], **decision["args"]}}
        return Command(goto="tools", update={"messages": [last.model_copy(update={"tool_calls":
[edited]})]})
    # reject: answer the tool call with an error, so the model knows and explains instead of retrying
    return Command(goto="call_model", update={"messages": [ToolMessage(
        f"REJECTED by merchant risk officer: {decision.get('note', 'no reason given')}",
tool_call_id=call["id"], status="error")]})

builder = StateGraph(State)
builder.add_node("call_model", call_model)
builder.add_node("approval", approval, destinations=("tools", "call_model"))   # where Command may go
(for the diagram)
builder.add_node("tools", ToolNode(ALL_TOOLS))
builder.add_edge(START, "call_model")
builder.add_conditional_edges("call_model", route_after_model, ["approval", "tools", END])
builder.add_edge("tools", "call_model")
```

3. Run, pause, decide, resume

You'll see `⏸ APPROVAL NEEDED` with the proposed arguments, and `paused before: ('approval',)`. Type `approve`, `edit` or `reject`.

hitl_run.py

```
# hitl_run.py — run until the graph pauses, show the request, decide, resume.
#   python hitl_run.py              # asks you: approve / edit / reject
# Resuming can happen hours later, in another process: the SQLite checkpoint holds everything.
import json
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.types import Command
from hitl_graph import builder

REQUEST = "ACME is in breach. Raise their limit to 10.5M USD for 5 days; the Black Friday sale starts tomorrow."

with SqliteSaver.from_conn_string("meridian.db") as checkpointer:
    graph = builder.compile(checkpointer=checkpointer)
    config = {"configurable": {"thread_id": "increase-acme-1"}, "recursion_limit": 25}

    out = graph.invoke({"messages": [("user", REQUEST)]}, config)
    if "__interrupt__" not in out:
        print("No approval needed:", out["messages"][-1].text)
        raise SystemExit

    request = out["__interrupt__"][0].value
    print("⏸ APPROVAL NEEDED\n" + json.dumps(request, indent=2))
    print("paused before:", graph.get_state(config).next)       # ('approval',)

    choice = input("approve / edit / reject? ").strip().lower()
    if choice == "edit":
        decision = {"type": "edit", "args": {"new_limit": 10_000_000, "days": 3}}
    elif choice == "reject":
        decision = {"type": "reject", "note": input("reason: ")}
    else:
        decision = {"type": "approve"}

    out = graph.invoke(Command(resume=decision), config)
    print("\nMeridian:", out["messages"][-1].text)
```

RUN

```
python hitl_run.py
```

4. Try all three decisions

Delete `meridian.db` between runs (or change the `thread_id`):

Decision	What happens	Check
approve	the tool runs with the model's arguments	limit becomes 10.5M
edit	the tool runs with *your* arguments (10M, 3 days)	Meridian reports 10M for 3 days
reject + reason	the tool never runs; the model gets `REJECTED: <reason>` and explains	limit stays 9.5M

5. **Restart test**

   Run until it pauses, then press **Ctrl+C**. Write a 5-line `resume.py` that compiles the same graph with the same checkpointer and calls `graph.invoke(Command(resume={"type": "approve"}), config)`. It finishes from where it stopped: the approval can come from a web app, a Slack button or a different server.

6. **Stretch: auto-approve small increases**

   Change `route_after_model` so increases of **≤5%** of the current limit skip the approval node. Write the rule so it reads the *current* limit with `lookup_merchant`, not the model's claim about it. Why does that matter?

Expected result. The run pauses with the increase details, then finishes according to your decision. After a reject, ACME's limit is still 9,500,000 and Meridian explains why it couldn't make the change.

LAB INTERMEDIATE 20 MIN

Lab 4: Memory across sessions

Checkpoints remember a conversation. A store remembers a person, across every conversation they have.

Goal. Meridian remembers an analyst's preference on Monday and uses it in a brand-new conversation on Tuesday.

1. The memory tools and graph

- The **store** is a key-value database with **namespaces**: here `("memories", user_id)`, so each analyst only ever sees their own memories.
- `user_id` comes from the run's config, *not* from the model. Never let the model choose whose memory it reads.
- `get_store()` / `get_config()` give tools access to the store and config of the current run.

memory_graph.py

```
# memory_graph.py — memory that lasts across conversations: Monday's preference is used on Tuesday.
from typing import Annotated
from typing_extensions import TypedDict
from dotenv import load_dotenv
from langchain_core.messages import AnyMessage, SystemMessage
from langchain_core.tools import tool
from langchain_openrouter import ChatOpenRouter
from langgraph.config import get_config, get_store
from langgraph.graph import START, StateGraph
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode, tools_condition
from tools_lc import TOOLS                          # Day 3 tools

load_dotenv()

def namespace():
    """Each analyst gets their own memory: ("memories", <user_id from the run config>)."""
    return ("memories", get_config()["configurable"]["user_id"])

@tool
def remember(fact: str) -> str:
    """Save a lasting preference or working fact about this analyst (e.g. preferred currency).
    Never store cardholder data, passwords or other personal data."""
    key = str(abs(hash(fact)))
    get_store().put(namespace(), key, {"fact": fact})
    return "saved"

@tool
def recall() -> list[str]:
    """List everything remembered about this analyst. Call it at the start of a conversation."""
    return [item.value["fact"] for item in get_store().search(namespace())]

class State(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]

ALL_TOOLS = TOOLS + [remember, recall]
llm = ChatOpenRouter(model="anthropic/claude-opus-5.5", max_tokens=8000).bind_tools(ALL_TOOLS)
SYSTEM = SystemMessage(
    "You are Meridian, a merchant-risk analyst at Northstar Pay. At the start of every conversation
call recall() "
    "and follow the analyst's saved preferences. When they state a lasting preference, save it with
```

```
remember()."
)

def call_model(state: State):
    return {"messages": [llm.invoke([SYSTEM, *state["messages"]])]}

builder = StateGraph(State)
builder.add_node("call_model", call_model)
builder.add_node("tools", ToolNode(ALL_TOOLS))
builder.add_edge(START, "call_model")
builder.add_conditional_edges("call_model", tools_condition)
builder.add_edge("tools", "call_model")
```

2. Monday, then Tuesday

Two different `thread_id`s means two separate conversations with no shared chat history. Only the store connects them.

memory_run.py

```
# memory_run.py — two conversations (threads), one analyst (user_id). Run:  python memory_run.py
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.store.sqlite import SqliteStore
from memory_graph import builder

with SqliteStore.from_conn_string("meridian_store.db") as store,
SqliteSaver.from_conn_string("meridian.db") as checkpointer:
    store.setup()                                  # creates the store's tables the first time
    graph = builder.compile(checkpointer=checkpointer, store=store)

    monday = {"configurable": {"thread_id": "mon-session", "user_id": "analyst-42"},
"recursion_limit": 25}
    out = graph.invoke({"messages": [("user", "Remember: always show MTD volumes in GBP as well as the
limit currency.")]}, monday)
    print("Monday:", out["messages"][-1].text[:200])

    # A new thread = a new conversation with no chat history. Only the store connects the two.
    tuesday = {"configurable": {"thread_id": "tue-session", "user_id": "analyst-42"},
"recursion_limit": 25}
    out = graph.invoke({"messages": [("user", "What's ACME's month-to-date volume?")]}, tuesday)
    print("Tuesday:", out["messages"][-1].text[:400])

    # What's stored, and for whom
    for item in store.search(("memories", "analyst-42")):
        print("stored:", item.value)
```

RUN `python memory_run.py`

3. Check what happened

Stream Tuesday's run with `stream_mode="updates"`: you should see `recall` called **before** the answer. Small local models sometimes skip it; Claude follows the system prompt reliably. If yours doesn't, make the first node of the graph call `recall` in code instead of asking the model to.

4. Semantic search (advanced)

Searching by meaning instead of by key: give the store an embedding model.

```
from langchain_ollama import OllamaEmbeddings
store = InMemoryStore(index={"embed": OllamaEmbeddings(model="nomic-embed-text"), "dims": 768,
```

```
"fields": ["fact"]})
store.search(("memories", "analyst-42"), query="which currency does she prefer?", limit=3)
```

The Postgres store supports the same `index` option (with pgvector) for production.

5. **Discuss memory governance**
   - **What may be stored long-term?** Preferences and working facts: yes. Card numbers, passwords, health data: never (put it in the tool's docstring *and* filter in code).
   - **How long?** `store.put(..., ttl=...)` (minutes) expires memories automatically. Pick a retention period you can explain.
   - **Delete on request:** `store.delete(namespace, key)`, or delete the whole namespace when someone leaves. GDPR requests must be answerable in minutes.

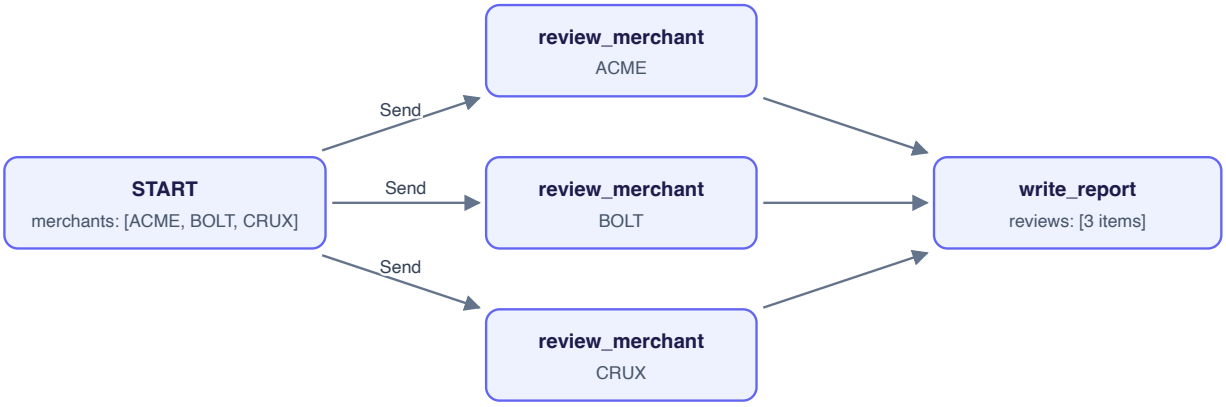
Expected result. On Tuesday, Meridian calls recall() first and shows ACME's volume with a GBP figure too. The store lists exactly one memory for analyst-42.

THEORY ADVANCED 10 MIN

Parallel work: Send, reducers and map-reduce

Reviewing 3 merchants one after another is slow and gets slower with 300. Fan the work out, then merge it.

Month-end review as map-reduce



1. A conditional edge returns a list of Send objects: one per merchant, each with its own input.
2. The three branches run in the same super-step, in parallel. Each returns {'reviews': [one result]}.
3. The reducer (operator.add) concatenates the three lists. Without it, the branches would overwrite each other.
4. write_report runs once, after every branch has finished: the 'reduce' step.

Piece	Code
Fan out	`builder.add_conditional_edges(START, lambda s: [Send("review_merchant", {"merchant_id": m}) for m in s["merchants"]], ["review_merchant"])`
Merge	`reviews: Annotated[list[dict], operator.add]`
Progress for a UI	`get_stream_writer()({"progress": "reviewing ACME"})` + `stream_mode="custom"`

Watch for: parallel branches that all call an LLM can hit rate limits at once. Limit concurrency with `graph.invoke(..., {"max_concurrency": 5})`.

Key points

- Any state key written by parallel branches needs a reducer.
- Send passes each branch its own input, not the whole state.

LAB ADVANCED 20 MIN

Lab 5: The month-end review, in parallel

No API key needed: the reviews are deterministic, so you can focus on how the graph runs.

Goal. Review every merchant in parallel with Send, merge the results with a reducer, and stream live progress.

1. Read and run

`month_end.py`

```
# month_end.py — review every merchant IN PARALLEL, then merge the results into one report.
# No API key needed: the review is deterministic. Run:  python month_end.py
import operator
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.config import get_stream_writer
from langgraph.graph import END, START, StateGraph
from langgraph.types import Send

MERCHANTS = {
    "ACME": {"limit": 9_500_000, "currency": "USD", "batches": [(4_200_000, "EUR"), (3_100_000,
"USD"), (2_000_000, "EUR")]},
    "BOLT": {"limit": 5_000_000, "currency": "GBP", "batches": [(1_500_000, "USD"), (2_000_000,
"GBP")]},
    "CRUX": {"limit": 800_000_000, "currency": "INR", "batches": [(650_000_000, "INR"), (2_000_000,
"USD")]},
}
FX = {("EUR", "USD"): 1.10, ("GBP", "USD"): 1.30, ("USD", "INR"): 85.0}

def rate(src: str, dst: str) -> float:
    if src == dst:
        return 1.0
    if (src, dst) in FX:
        return FX[(src, dst)]
    return 1 / FX[(dst, src)]                    # e.g. USD -> GBP from the GBP -> USD rate

class MonthEnd(TypedDict):
    merchants: list[str]
    # Reducer: each parallel branch RETURNS a one-item list; operator.add concatenates them.
    # Without it, branches would overwrite each other's results.
    reviews: Annotated[list[dict], operator.add]
    report: str

class ReviewInput(TypedDict):                    # what each parallel branch receives
    merchant_id: str

def fan_out(state: MonthEnd):
    """One Send per merchant: LangGraph runs these branches in parallel (a 'map')."""
    return [Send("review_merchant", {"merchant_id": m}) for m in state["merchants"]]

def review_merchant(task: ReviewInput):
    write = get_stream_writer()                  # custom progress events for a UI
    m_id = task["merchant_id"]
    m = MERCHANTS[m_id]
    write({"progress": f"reviewing {m_id}"})
    volume = sum(amount * rate(cur, m["currency"]) for amount, cur in m["batches"])
    pct = volume / m["limit"] * 100
    verdict = "OVER_LIMIT" if pct > 100 else "NEAR_LIMIT" if pct > 90 else "OK"
    write({"progress": f"{m_id} done: {verdict}"})
    return {"reviews": [{"merchant": m_id, "volume": round(volume), "currency": m["currency"], "pct":
round(pct, 1), "verdict": verdict}]}
```

```
def write_report(state: MonthEnd):
    """The 'reduce': runs once, after every branch has finished."""
    rows = sorted(state["reviews"], key=lambda r: -r["pct"])
    lines = ["merchant  volume (limit ccy)        used   verdict"]
    lines += [f"{r['merchant']:<8}  {r['volume']:>16,} {r['currency']}  {r['pct']:>6}%
{r['verdict']}" for r in rows]
    return {"report": "\n".join(lines)}

builder = StateGraph(MonthEnd)
builder.add_node("review_merchant", review_merchant)
builder.add_node("write_report", write_report)
builder.add_conditional_edges(START, fan_out, ["review_merchant"])
builder.add_edge("review_merchant", "write_report")
builder.add_edge("write_report", END)
graph = builder.compile()

if __name__ == "__main__":
    # stream_mode="custom" shows our progress events; "updates" shows which node produced what.
    for mode, chunk in graph.stream({"merchants": list(MERCHANTS)}, stream_mode=["custom",
"updates"]):
        if mode == "custom":
            print("  ·", chunk["progress"])
        elif "write_report" in chunk:
            print("\n" + chunk["write_report"]["report"])
```

RUN `python month_end.py`

2. Notice the order

Run it three times. The progress lines can come in a different order each time, but the report is always sorted. Parallel branches finish when they finish; never rely on their order.

3. Remove the reducer

Change `reviews: Annotated[list[dict], operator.add]` to `reviews: list[dict]` and run. LangGraph raises **InvalidUpdateError**: several branches wrote the same key in one step, and it refuses to guess which one wins. Put the reducer back.

4. Scale it

Add 50 fake merchants to `MERCHANTS` (copy ACME with new IDs in a loop) and time the run. Then add `time.sleep(0.2)` in `review_merchant` and compare `max_concurrency` 1, 10 and 50 in the config.

5. Stretch: an LLM summary

Add a `summarise` node after `write_report` that asks Claude for a 3-sentence risk-committee summary of the report. Keep the numbers out of the model's hands: pass it the table, and tell it not to recalculate.

Expected result. ACME 9,920,000 USD 104.4% OVER_LIMIT; CRUX 820,000,000 INR 102.5% OVER_LIMIT; BOLT 3,153,846 GBP 63.1% OK, with progress lines printed as each branch runs.

THEORY ADVANCED 10 MIN

Durable execution: retries, timeouts, caching, resuming

Real agents call flaky services, run for minutes, and get restarted by deploys. A graph with a checkpointer can survive all of that.

Problem	LangGraph feature	Example
A service fails now and then	`retry_policy=RetryPolicy(max_attempts=5, retry_on=ConnectionError)` on the node	FX provider times out
A call hangs	`timeout=30` on an **async** node (sync nodes can't be cancelled)	a slow bank API
Same expensive call again	`cache_policy=CachePolicy(ttl=300)` + `compile(cache=InMemoryCache())`	today's FX rate
The process dies mid-run	checkpointer + `invoke(None, config)` resumes at the next unfinished node	deploy during month-end
How safe is "saved"?	`durability="sync"` (save before each next step), `"async"` (default), `"exit"` (only at the end)	money movement → `"sync"`

Idempotency matters: after a crash, the node that was running runs **again**. A payout node must be safe to repeat: check "already paid?" with a unique key before paying.

Key points

- Retries are for transient errors only: retry a timeout, never a 'card declined'.
- Resume = invoke(None, same thread_id).

LAB ADVANCED 20 MIN

Lab 6: A settlement run that survives a crash

No API key needed. You'll watch retries happen, then kill the run and resume it without redoing finished work.

Goal. Retry a flaky FX service with backoff, crash during settlement, and resume from the last checkpoint.

1. Read and run once

The first two FX calls fail and are retried (attempts 1–3), then the run crashes in `settle`.

durable.py

```
# durable.py — a settlement run that survives a flaky FX service AND a crash half-way through.
# No API key needed. Run it twice:  python durable.py   (crashes at "settle")  then  python durable.py
(resumes)
import sqlite3
from typing_extensions import TypedDict
from langgraph.cache.memory import InMemoryCache
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.graph import END, START, StateGraph
from langgraph.types import CachePolicy, RetryPolicy

calls = {"fx": 0}

class Run(TypedDict, total=False):
    merchant: str
    amount_eur: float
    eur_usd: float
    amount_usd: float
    settled: str

def fetch_fx(state: Run):
    """A flaky upstream service: the first 2 calls time out. RetryPolicy retries with backoff (0.2s,
0.4s, ...)."""
    calls["fx"] += 1
    print(f"  fetch_fx attempt {calls['fx']}")
    if calls["fx"] <= 2:
        raise ConnectionError("FX provider timed out")
    return {"eur_usd": 1.10}

def convert(state: Run):
    return {"amount_usd": round(state["amount_eur"] * state["eur_usd"], 2)}

def settle(state: Run):
    """Pretend the process crashes here the first time (a deploy, an OOM kill, a laptop lid
closed)."""
    import os
    if not os.path.exists("settle.ok"):
        open("settle.ok", "w").close()
        raise SystemExit("💥 crashed while settling (run the script again to resume)")
    return {"settled": f"{state['merchant']}: paid out {state['amount_usd']:,} USD"}

builder = StateGraph(Run)
builder.add_node(
    "fetch_fx", fetch_fx,
    retry_policy=RetryPolicy(max_attempts=6, initial_interval=0.2, backoff_factor=2.0,
retry_on=ConnectionError),
    cache_policy=CachePolicy(ttl=300),          # same inputs within 5 minutes -> reuse the result
)
```

```
builder.add_node("convert", convert)
builder.add_node("settle", settle)
builder.add_edge(START, "fetch_fx")
builder.add_edge("fetch_fx", "convert")
builder.add_edge("convert", "settle")
builder.add_edge("settle", END)

conn = sqlite3.connect("durable.sqlite", check_same_thread=False)
graph = builder.compile(checkpointer=SqliteSaver(conn), cache=InMemoryCache())
config = {"configurable": {"thread_id": "settlement-ACME-2026-10"}}

if __name__ == "__main__":
    state = graph.get_state(config)
    if state.values.get("settled"):                  # this thread already finished
        print("already settled:", state.values["settled"])
        print("delete durable.sqlite and settle.ok to start over")
        raise SystemExit
    if state.next:                                   # an unfinished run exists for this thread:
resume it
        print(f"resuming at {state.next} (fetch_fx and convert are NOT run again)")
        result = graph.invoke(None, config, durability="sync")
    else:
        print("new run")
        result = graph.invoke({"merchant": "ACME", "amount_eur": 6_200_000}, config,
durability="sync")
    print(result["settled"])
    print(f"FX service was called {calls['fx']} time(s) in this process")
```

RUN `python durable.py`

2. Resume

The second run sees an unfinished thread and resumes at `('settle',)`. **The FX service is called 0 times**: its result was in the checkpoint.

RUN `python durable.py`

3. Run it a third time

It reports the run is already settled instead of paying again. That check is your idempotency guard. Delete `durable.sqlite` and `settle.ok` to start over.

4. Change the retry rule

Set `max_attempts=2` and start over. The run fails with **ConnectionError** after 2 attempts. Then set `retry_on=ValueError` (the wrong exception): it fails at once. Retries must match the error you expect.

5. Stretch: a timeout

Make `fetch_fx` `async def` with `await asyncio.sleep(5)`, add `timeout=1` to its `add_node`, and run with `await graph.ainvoke(...)`. You get **NodeTimeoutError**, which the retry policy can also retry.

Expected result. Run 1: three FX attempts, then the crash. Run 2: 'resuming at ('settle',)', ACME paid out 6,820,000 USD, FX called 0 times. Run 3: 'already settled'.

THEORY ADVANCED 5 MIN

Time travel: replay and fork

Because every step is saved, you can go back to any point, change something, and run forward on a new branch.

- `graph.get_state_history(config)` : every checkpoint, newest first. Each has `values`, `next` and its own `config`.
- `graph.invoke(None, snapshot.config)` : **replay** from that point.
- `graph.update_state(snapshot.config, {...}, as_node="...")` : **fork**, i.e. write a change into that checkpoint as if a node had produced it, then continue from the new branch.

Uses: debugging ("what did it see before the wrong call?"), what-if analysis for risk ("and with a 10.5M limit?"), and fixing a bad tool result without rerunning the whole conversation.

LAB ADVANCED 15 MIN

Lab 7: What if the limit were higher?

Goal. Inspect a run's history, then fork it with a different limit and compare both outcomes.

1. Read and run

time_travel.py

```
# time_travel.py — every step of a checkpointed run is saved. Inspect the history, then FORK it:
# "what would the verdict have been if ACME's limit were 10.5M?" No API key needed.  Run:  python
time_travel.py
from typing_extensions import TypedDict
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph import END, START, StateGraph

class Check(TypedDict, total=False):
    merchant: str
    volume_usd: float
    limit_usd: float
    pct: float
    verdict: str

def load_limit(state: Check):
    return {"limit_usd": 9_500_000}                      # from the merchant file

def compute(state: Check):
    return {"pct": round(state["volume_usd"] / state["limit_usd"] * 100, 1)}

def decide(state: Check):
    p = state["pct"]
    return {"verdict": "OVER_LIMIT" if p > 100 else "NEAR_LIMIT" if p > 90 else "OK"}

builder = StateGraph(Check)
for name, fn in [("load_limit", load_limit), ("compute", compute), ("decide", decide)]:
    builder.add_node(name, fn)
builder.add_edge(START, "load_limit")
builder.add_edge("load_limit", "compute")
builder.add_edge("compute", "decide")
builder.add_edge("decide", END)
graph = builder.compile(checkpointer=InMemorySaver())

config = {"configurable": {"thread_id": "acme-review"}}
graph.invoke({"merchant": "ACME", "volume_usd": 9_920_000}, config)
print("original verdict:", graph.get_state(config).values["verdict"])

# 1. The history: one checkpoint per step, newest first. `next` is what would run from there.
print("\nhistory (newest first):")
history = list(graph.get_state_history(config))
for snap in history:
    print(f"  step {snap.metadata['step']:>2}  next={snap.next or '(done)'}  limit=
{snap.values.get('limit_usd')}")

# 2. Pick the checkpoint just before `compute` ran: the limit is loaded, nothing calculated yet.
before_compute = next(s for s in history if s.next == ("compute",))

# 3. Fork: write a different limit into THAT checkpoint (as if load_limit had returned it), then
continue from it.
fork_config = graph.update_state(before_compute.config, {"limit_usd": 10_500_000},
as_node="load_limit")
forked = graph.invoke(None, fork_config)                 # None = "continue from this checkpoint"
print(f"\nfork verdict (limit 10.5M): {forked['verdict']} ({forked['pct']}%)")
```

```
# 4. The original branch is untouched; the thread's latest state is now the fork.
print("original still:", history[0].values["verdict"], "| latest in thread:",
graph.get_state(config).values["verdict"])
print("checkpoints in the thread now:", len(list(graph.get_state_history(config))))
```

RUN `python time_travel.py`

2. Read the history

Each line is a checkpoint. `next` is what would run if you resumed there. Step -1 is the input; the last one has `next=(done)`.

3. Fork from a different point

Fork from the checkpoint before `decide` instead, and set `pct` directly to `91.0`. What verdict do you get? Which `as_node` do you need, and why?

4. Use it on Lab 3

In the approval graph's history, find the checkpoint where the model proposed 10.5M. Fork it with an edited tool call (8% instead of 10.5%) and continue. This is how you'd fix a bad proposal without restarting the conversation.

Expected result. Original OVER_LIMIT; the fork with a 10.5M limit gives NEAR_LIMIT at 94.5%; both branches stay in the thread's history.

THEORY ADVANCED 5 MIN

Taking a graph to production

Concern	Do this
Persistence	`PostgresSaver` and `PostgresStore` (one database, backed up). SQLite is for laptops
Many users	one `thread_id` per conversation, `user_id` in config for memory namespaces; never trust IDs from the model
Approvals in a web app	API endpoint returns the `__interrupt__` payload; the approver's button calls `invoke(Command(resume=...))`
Long runs	stream progress (`"custom"`), `durability="sync"` for money movement, idempotent side effects
Limits	`recursion_limit`, `max_concurrency`, node `timeout`s, and a token/cost budget (Day 8)
Audit	keep checkpoints for your retention period; they show exactly what the agent knew at each step
Testing	unit-test nodes as plain functions; test routing with fake models (`GenericFakeChatModel`)

QUIZ 5 MIN

Check yourself

1. **What does a reducer do?**
   A. Shrinks the prompt
   B. Decides how a node's update merges into the state (e.g. append messages)
   C. Removes tools
   D. Compresses the database

2. **What two things does interrupt() need?**
   A. A GPU and a key
   B. A checkpointer and a thread_id
   C. A store and a namespace
   D. Two models

3. **Short-term vs long-term memory in LangGraph?**
   A. Both are the store
   B. Checkpointer (per thread) vs store (across threads)
   C. RAM vs disk
   D. Prompt vs model

4. **Why must side effects come after interrupt()?**
   A. Style
   B. On resume the node re-runs from the top, so earlier side effects would run twice
   C. They're faster
   D. LangGraph forbids them

5. **Rule: increases ≤5% are auto-approved. Prompt or code?**
   A. Prompt
   B. Code (an edge / condition)

6. **Three parallel branches each write to `reviews` in the same step. What do you need?**
   A. Nothing
   B. A reducer on `reviews` (e.g. operator.add)
   C. A bigger recursion_limit
   D. A checkpointer

7. **A run crashed in `settle` after `fetch_fx` succeeded. How do you resume?**
   A. Run the whole thing again
   B. graph.invoke(None, config) with the same thread_id
   C. Delete the checkpoint
   D. Call settle() directly

8. **Which errors should a RetryPolicy retry?**
   A. All of them
   B. Transient ones: timeouts, connection errors, 5xx
   C. Validation errors
   D. Card declined

9. **update_state(old_checkpoint_config, {...}) does what?**
    A. Overwrites history
    B. Creates a new branch from that point, leaving the original intact
    C. Deletes the thread
    D. Restarts the graph

RECAP 3 MIN

Day 4 recap

- A graph makes the agent loop explicit: nodes, edges, cycles.
- Reducers merge updates; memory is working, short-term (checkpointer) and long-term (store).
- Checkpoints make runs resumable and auditable.
- Human approval with interrupt(): approve, edit or reject, enforced in code.
- Send + reducers run work in parallel; retries, caching and checkpoints make runs durable; history lets you replay and fork.

Next: Day 5: Meridian learns to read Northstar's policies, and to notice when a search came back wrong.

Quiz answer key

1. **B. Decides how a node's update merges into the state (e.g. append messages)**
   add_messages appends; without a reducer, the last write wins.
2. **B. A checkpointer and a thread_id**
   The state must be saved, and the resume must find the same thread.
3. **B. Checkpointer (per thread) vs store (across threads)**
   Threads are sessions; the store survives across them.
4. **B. On resume the node re-runs from the top, so earlier side effects would run twice**
   Resume re-executes the node.
5. **B. Code (an edge / condition)**
   Business rules belong in deterministic code, where they can't be talked around.
6. **B. A reducer on `reviews` (e.g. operator.add)**
   Without a reducer LangGraph can't merge concurrent writes and raises InvalidUpdateError.
7. **B. graph.invoke(None, config) with the same thread_id**
   The checkpoint holds fetch_fx's result; invoking with None continues at the next unfinished node.
8. **B. Transient ones: timeouts, connection errors, 5xx**
   Retrying a permanent failure just repeats it (and can double-charge if the step isn't idempotent).
9. **B. Creates a new branch from that point, leaving the original intact**
   Forking keeps both branches; the thread's latest state becomes the fork.